SOLID Principles

Five simple Java examples — before and after

JAVA | OOP DESIGN

S Single Responsibility Principle

Definition: A class should have only **one reason to change** — it should do only one job.

Problem: This class calculates the invoice, prints it, and saves it to the database. That is three responsibilities in one class.

BEFORE (VIOLATES SRP)

```
class Invoice {
    double calculateTotal(List<Item> items) {
        double total = 0;
        for (Item i : items) total += i.getPrice();
        return total;
    }

    void printInvoice() {
        System.out.println("Printing invoice...");
    }

    void saveToDatabase() {
        System.out.println("Saving invoice to DB...");
    }
}
```

Solution: Split each job into its own class. Each class now has only one reason to change.

AFTER (FOLLOWS SRP)

```
class Invoice {
    double calculateTotal(List<Item> items) {
        double total = 0;
        for (Item i : items) total += i.getPrice();
        return total;
    }
}

class InvoicePrinter {
    void print(Invoice invoice) {
        System.out.println("Printing invoice...");
    }
}

class InvoiceRepository {
    void save(Invoice invoice) {
        System.out.println("Saving invoice to DB...");
    }
}
```

Takeaway: If you must describe a class with "and", it probably breaks SRP. Split it.

O Open/Closed Principle

Definition: Classes should be **open for extension** but **closed for modification** — add new behavior without changing existing code.

Problem: Every time a new shape is added, we must edit this method and add another `if`.

BEFORE (VIOLATES OCP)

```
class AreaCalculator {
    double calculate(Object shape) {
        if (shape instanceof Circle) {
            Circle c = (Circle) shape;
            return Math.PI * c.radius * c.radius;
        } else if (shape instanceof Square) {
            Square s = (Square) shape;
            return s.side * s.side;
        }
        return 0;
    }
}
```

Solution: Use a common interface. New shapes just implement it — no existing code changes.

AFTER (FOLLOWS OCP)

```
interface Shape {
    double area();
}

class Circle implements Shape {
    double radius;
    public double area() { return Math.PI * radius * radius; }
}

class Square implements Shape {
    double side;
    public double area() { return side * side; }
}

class AreaCalculator {
    double calculate(Shape shape) {
        return shape.area(); // works for any shape, even future ones
    }
}
```

Takeaway: A long chain of `if/instanceof` is a warning sign. Prefer interfaces + polymorphism.

L Liskov Substitution Principle

Definition: A subclass should be **usable anywhere its parent class is used**, without breaking the program.

Problem: `Ostrich` is a `Bird` but cannot fly. Calling `fly()` on it breaks the program.

BEFORE (VIOLATES LSP)

```
class Bird {
    void fly() {
        System.out.println("Flying...");
    }
}

class Ostrich extends Bird {
    void fly() {
        throw new UnsupportedOperationException("Ostriches can't fly!");
    }
}
```

Solution: Only birds that can fly implement a `Flyable` behavior. `Ostrich` no longer promises something it can't do.

AFTER (FOLLOWS LSP)

```
class Bird {
    // common bird behavior, no fly() here
}

interface Flyable {
    void fly();
}

class Sparrow extends Bird implements Flyable {
    public void fly() {
        System.out.println("Flying...");
    }
}

class Ostrich extends Bird {
    // no fly() method — and no broken promise
}
```

Takeaway: If a subclass has to throw an exception to "opt out" of a method, the hierarchy is wrong.

I Interface Segregation Principle

Definition: Don't force a class to implement methods it doesn't need — prefer several **small interfaces** over one big one.

Problem: `Robot` is forced to implement `eat()`, even though robots don't eat.

BEFORE (VIOLATES ISP)

```
interface Worker {
    void work();
    void eat();
}

class Robot implements Worker {
    public void work() {
        System.out.println("Robot working...");
    }
    public void eat() {
        throw new UnsupportedOperationException("Robots don't eat!");
    }
}
```

Solution: Split into two small interfaces. Each class only implements what it actually needs.

AFTER (FOLLOWS ISP)

```
interface Workable {
    void work();
}

interface Eatable {
    void eat();
}

class Robot implements Workable {
    public void work() {
        System.out.println("Robot working...");
    }
}

class Human implements Workable, Eatable {
    public void work() { System.out.println("Human working..."); }
    public void eat()  { System.out.println("Human eating..."); }
}
```

Takeaway: A method that throws "not supported" is usually a sign the interface is too fat.

D Dependency Inversion Principle

Definition: Classes should depend on **abstractions (interfaces)**, not on concrete/low-level classes.

Problem: `NotificationService` is hard-wired to `EmailSender`. Adding SMS later means changing this class.

BEFORE (VIOLATES DIP)

```
class EmailSender {
    void send(String message) {
        System.out.println("Email sent: " + message);
    }
}

class NotificationService {
    private EmailSender sender = new EmailSender();

    void notify(String message) {
        sender.send(message);
    }
}
```

Solution: Depend on a `MessageSender` interface. Any implementation (email, SMS, push) can be plugged in.

AFTER (FOLLOWS DIP)

```
interface MessageSender {
    void send(String message);
}

class EmailSender implements MessageSender {
    public void send(String message) {
        System.out.println("Email sent: " + message);
    }
}

class SmsSender implements MessageSender {
    public void send(String message) {
        System.out.println("SMS sent: " + message);
    }
}

class NotificationService {
    private final MessageSender sender;

    NotificationService(MessageSender sender) { // injected
        this.sender = sender;
    }

    void notify(String message) {
        sender.send(message);
    }
}
```

Takeaway: "new" inside a class is a warning sign. Pass dependencies in from outside (constructor injection).